

# Microsoft Teams + Copilot Studio integration playbook

---

This playbook describes the next implementation step after validating the IsPilot backend API.

## Goal

Expose the validated IsPilot API through Microsoft Copilot Studio and then Microsoft Teams without moving the business logic into the channel layer.

## Architecture

### Option 1: Copilot Studio + Custom Connector (Recommended for Teams/enterprise)

```
User in Teams / Copilot Studio
  ↓
Custom Connector
  ↓
Cloud Run API (ispilot-api)
  ↓
Vertex AI Reasoning Engine
  ↓
Coordinator + Specialist Agents
  ↓
Business answers
```

### Option 2: Direct Teams Bridge Webhook (Current implementation)

The direct Teams Bridge approach provides a lightweight, direct webhook pattern:

```
User in Teams
  ↓
Teams Webhook → Cloud Run (teams-ispilot-bridge) [PUBLIC]
  ↓
[Teams Bridge generates identity token using sa-tot-osa]
  ↓
Cloud Run (ispilot-api) [PRIVATE]
  ↓
Vertex AI Reasoning Engine
  ↓
Response back through Teams
```

### Architecture Diagram:

- Teams sends unauthenticated HTTP POST to **teams-ispilot-bridge** webhook

- Teams Bridge validates the request, generates an identity token
- Teams Bridge calls `ispilot-api` with the token in `Authorization: Bearer {token}` header
- IsPilot API validates the token and processes the request
- Response flows back through the bridge to Teams

### When to use Teams Bridge:

- Direct integration with Teams without Copilot Studio
- Simpler deployment (single service endpoint)
- Fast iteration on Teams bot scenarios

### When to use Copilot Studio:

- Enterprise channel integration (Teams, Outlook, etc.)
- Advanced routing and branching logic needed
- Managed lifecycle and versioning desired

### Required IAM Setup for Teams Bridge:

See [Teams Bridge README](#) for complete IAM configuration steps.

## Production contract

Use the validated API contract from:

- `ispilot-api/openapi.yaml`

This API currently exposes:

- `GET /health`
- `POST /chat`

The production endpoint is:

```
https://ispilot-api-46y2f3tyja-uc.a.run.app
```

## Required auth pattern

Production bearer token

```
TOKEN=$(gcloud auth print-identity-token \
  --impersonate-service-account=sa-tot-osa@corp-stro-salesinventory-
  prod.iam.gserviceaccount.com)
```

## Example request

```
curl -sS -X POST "https://ispilot-api-46y2f3tyja-uc.a.run.app/chat" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer ${TOKEN}" \
--data '{"user_id":"sebastian","message":"How is Talca Colin performing?"}'
```

## Example response

```
{
  "answer": "Talca Colín is performing at 95.16% SNSG, with 0.67% Bodega and 4.17% Quiebre.",
  "session_id": "5411387247947677696",
  "request_id": "req_550e8400-e29b-41d4-a716-446655440000",
  "timestamp": "2026-08-29T03:38:06.750012",
  "status": "ok"
}
```

## Connector setup in Copilot Studio

1. Create a new custom connector.
2. Import the OpenAPI file from `ispilot-api/openapi.yaml`.
3. Configure authentication:
  - preferred: Microsoft Entra ID or enterprise approved identity flow
  - backend remains Cloud Run, not direct Vertex access
4. Create an action for `POST /chat`.
5. Map the request body to:
  - `user_id`
  - `message`
  - `session_id` (optional)
6. Test the connector with a real business question.

## Recommended action schema

```
{
  "user_id": "{{user.id}}",
  "message": "{{user_message}}",
  "session_id": null
}
```

For follow-up questions, preserve the `session_id` returned by the API.

## Validation scenarios

Test these prompts as the first pass in Copilot Studio and Teams:

1. "How is Talca Colin performing?"
2. "How can Talca Colin improve?"
3. "What are today's priorities?"
4. "Which stores need immediate attention?"

Expected result:

- user question enters Copilot Studio or Teams
- connector calls the Cloud Run API
- API calls the validated IsPilot reasoning engine
- answer is returned with business context and a clear operational recommendation

## Teams rollout plan

1. Validate the connector in Copilot Studio.
2. Publish the connector to the target team or environment.
3. Add the app or bot in Microsoft Teams.
4. Confirm the user identity and session lifecycle are handled correctly.
5. Monitor logs and request IDs in the backend.

## Production guardrails

- Do not expose the Vertex Engine ID or internal routing logic to end users.
- Do not call the reasoning engine directly from Copilot Studio.
- Keep the API as the single integration boundary.
- Treat the Cloud Run service as the authoritative production endpoint.

## Exit criteria

The integration is ready when all of the following are true:

- the custom connector calls the live API successfully
- a real business question returns a valid answer
- the response is visible in Copilot Studio
- the response is also consumable by Teams
- the request\_id and session\_id are traceable in logs

## Recommended next implementation task

Start with the custom connector in Copilot Studio using the provided OpenAPI file and validate the first prompt in a test environment before publish to Teams.